Ejemplo de bit de paridad

P_R	B_6	B_5	B_4	B_3	B_2	B_1	B_0	
0	0	0	0	0	0	1	0	STX
1	0	1	0	1	0	0	0	
0	1	0	0	0	1	1	0	
0	0	1	0	0	0	0	0	Contenido de la trama
1	0	1	0	1	1	0	1	
0	1	0	0	0	0	0	0	
1	1	1	0	0	0	1	1	
1	0	0	0	0	0	1	1	ETX



Bits de Paridad

Si se produce un error lo detectamos, pero no lo podemos corregir
Si se producen dos errores en una misma fila no los detectamos

Ejemplo simplificado del funcionamiento del CRC: parte 1 (transmisión)

- El nivel de red le entrega al nivel de enlace un paquete con los datos, concretamente el número **302**
- El nivel de enlace ha acordado con su homólogo en el otro extremo utilizar un CRC de un dígito, y como generador el número **3**. Por tanto sabe que tiene que enviar **302x**
- Elige ***x*** tal que el número resultante sea divisible por 3. Tomará la primera posibilidad:

  prueba con 0 →3020, no es divisible

  Prueba con 1 →3021, este sí
- El nivel de enlace transmite una trama que contiene **3021**

Ejemplo simplificado del funcionamiento del CRC: parte 2 (recepción)

- El nivel de enlace en el receptor recibe **3021**. Sabe que el último dígito (el 1) es el CRC y que el generador acordado es 3.
- Divide 3021 por 3 y comprueba que el resultado es entero.
- Da por buena la trama y entrega **302** al nivel de red

Ejemplo de código corrector con bits de paridad

P_R	B_6	B_5	B_4	B_3	B_2	B_1	B_0	
0	0	0	0	0	0	1	0	STX
1	0	1	0	1	0	0	0	
0	1	0	0	0	1	1	0	
0	0	1	0	0	0	0	0	
1	0	1	0	1	1	0	1	
0	1	0	0	0	0	0	0	
1	1	1	0	0	0	1	1	
1	0	0	0	0	0	1	1	ETX
1	1	0	0	0	0	0	1	

Bits de Paridad transversales

Contenido de la trama

Bits de Paridad longitudinales

En este caso el uso de dos bits de paridad nos permite detectar el bit que ha fallado y corregirlo

t_trans = t_propag

Caudal	T_trans 540 bytes	T_propag equivalente	
		Cobre-Fibra	WiFi
40 Gb/s	0,1 µs	22 m	32 m
10 Gb/s	0,4 µs	86 m	130 m
1 Gb/s	4,3 µs	860 m	1300 m
400 Mb/s	10,8 µs	2,2 Km	3,2 Km
200 Mb/s	21,6 µs	4,3 Km	6,5 Km
100 Mb/s	43,2 µs	8,6 Km	13 Km
50 Mb/s	86,4 µs	17 Km	26 Km
10 Mb/s	432 µs	86 Km	130 Km

La distancia indicada en la tabla es la que iguala t_trans y t_propag. Para distancias superiores t_propag resulta dominante, para distancias inferiores lo es t_trans. Valores calculados para paquetes de 540 bytes

Tiempo de ida y vuelta

- También llamado tiempo RTT (Round Trip Time) o 'tiempo del ping'. Es el tiempo que tarda en recibirse la respuesta a un paquete. Corresponde a la latencia en sentido de ida y vuelta, más el tiempo de proceso en el host de destino
- t_trans puede ser diferente a la ida y a la vuelta si los paquetes son de distinto tamaño o si el enlace es de caudal asimétrico (ADSL por ejemplo)
- t_propag puede ser diferente a la ida y a la vuelta si la ruta es asimétrica
- El tiempo de proceso es difícil de estimar y depende de la carga en el host de destino, pero a menudo se considera despreciable.

Cálculo de distancias a partir del RTT



Van Jacobson
(1950-)

- traceroute (creado por Van Jacobson) permite descubrir, mediante el envío de paquetes sonda pequeños, los nodos de tránsito a nivel 3 hacia un destino cualquiera.
- El programa devuelve además el RTT hasta cada uno de los nodos intermedios
- Con paquetes pequeños y distancias grandes el factor predominante de RTT es t_propag. De los valores obtenidos se puede averiguar la longitud de los enlaces en rutas de larga distancia (como ocurre con el tren o las carreteras la ruta de los datos no suele seguir una línea recta).

```
C:\Documents and Settings\montanan>ping www.cern.ch

Haciendo ping a webr2.cern.ch [137.138.28.230] con 32 bytes de datos:

Respuesta desde 137.138.28.230: bytes=32 tiempo=43ms TTL=114
Respuesta desde 137.138.28.230: bytes=32 tiempo=43ms TTL=114
Respuesta desde 137.138.28.230: bytes=32 tiempo=43ms TTL=114
Respuesta desde 137.138.28.230: bytes=32 tiempo=43ms TTL=114

Estadísticas de ping para 137.138.28.230:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0 (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 43ms, Máximo = 43ms, Media = 43ms


C:\Documents and Settings\montanan >tracert www.cern.ch

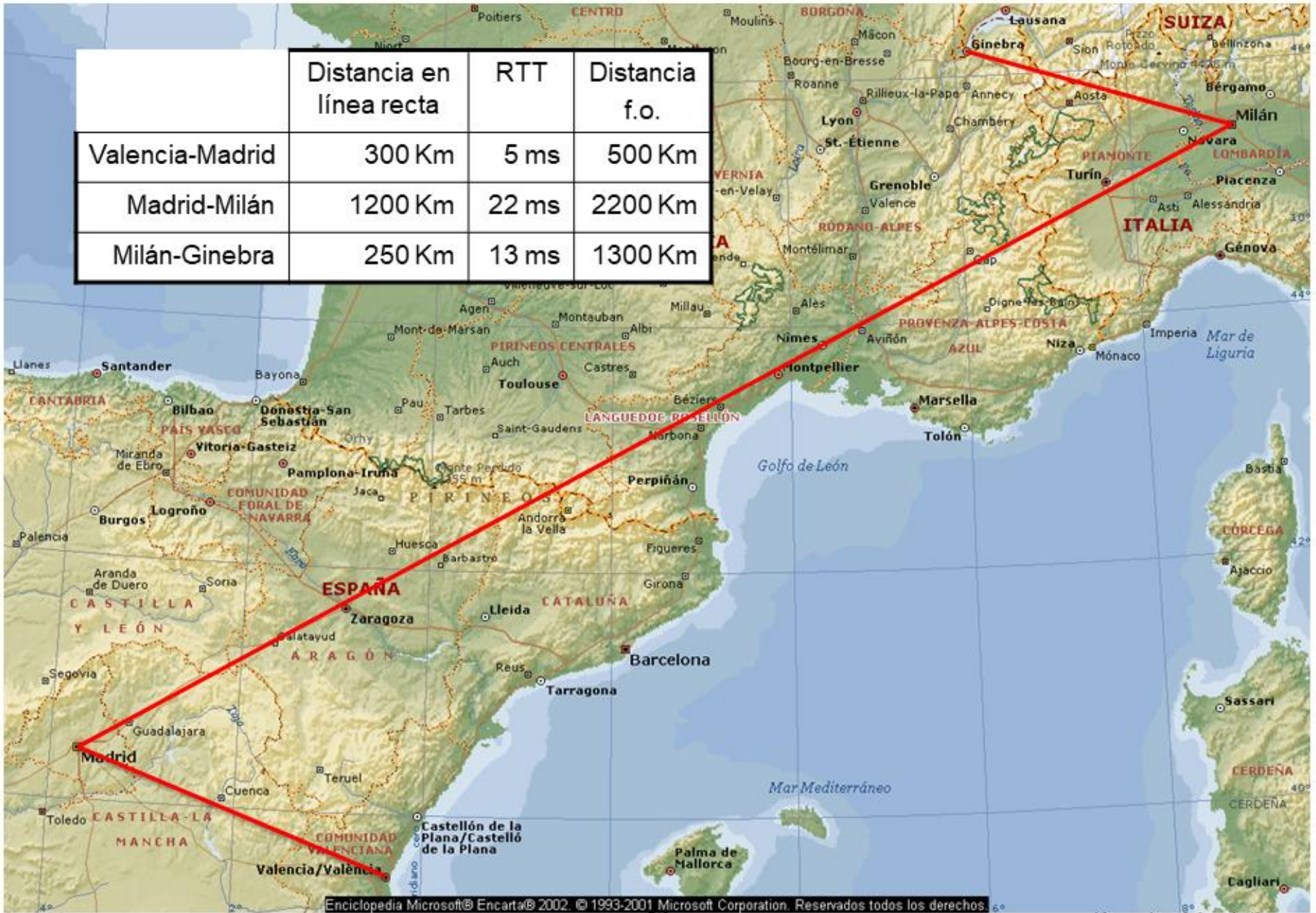
Traza a la dirección webr2.cern.ch [137.138.28.230] sobre un máximo de 30 saltos:

  1    <1 ms    <1 ms    <1 ms  burl3ci.red.uv.es [147.156.2.50]
  2    <1 ms    <1 ms    <1 ms  burladerol3.red.uv.es [147.156.200.184]
  3    <1 ms    <1 ms    <1 ms  neveraout.red.uv.es [147.156.7.1]
  4    <1 ms    <1 ms    <1 ms  AT0-2-0-0.EB-Valencia0.red.rediris.es [130.206.211.181]
  5     6 ms     6 ms     6 ms  VAL.SO3-0-0.EB-IRIS4.red.rediris.es [130.206.240.13]
  6     7 ms     7 ms     7 ms  rediris.es1.es.geant.net [62.40.103.61]
  7    29 ms    29 ms    29 ms  es.it1.it.geant.net [62.40.96.186]
  8    42 ms    42 ms    42 ms  it.ch1.ch.geant.net [62.40.96.33]
  9    43 ms    42 ms    42 ms  swiCE2-P6-1.switch.ch [62.40.103.18]
 10    43 ms    43 ms    42 ms  cernh7-gb1-1.cern.ch [192.65.184.222]
 11    42 ms    43 ms    42 ms  cernh2-vlan2.cern.ch [192.65.192.2]
C:\Documents and Settings\montanan>
```

5 ms

22 ms

13 ms

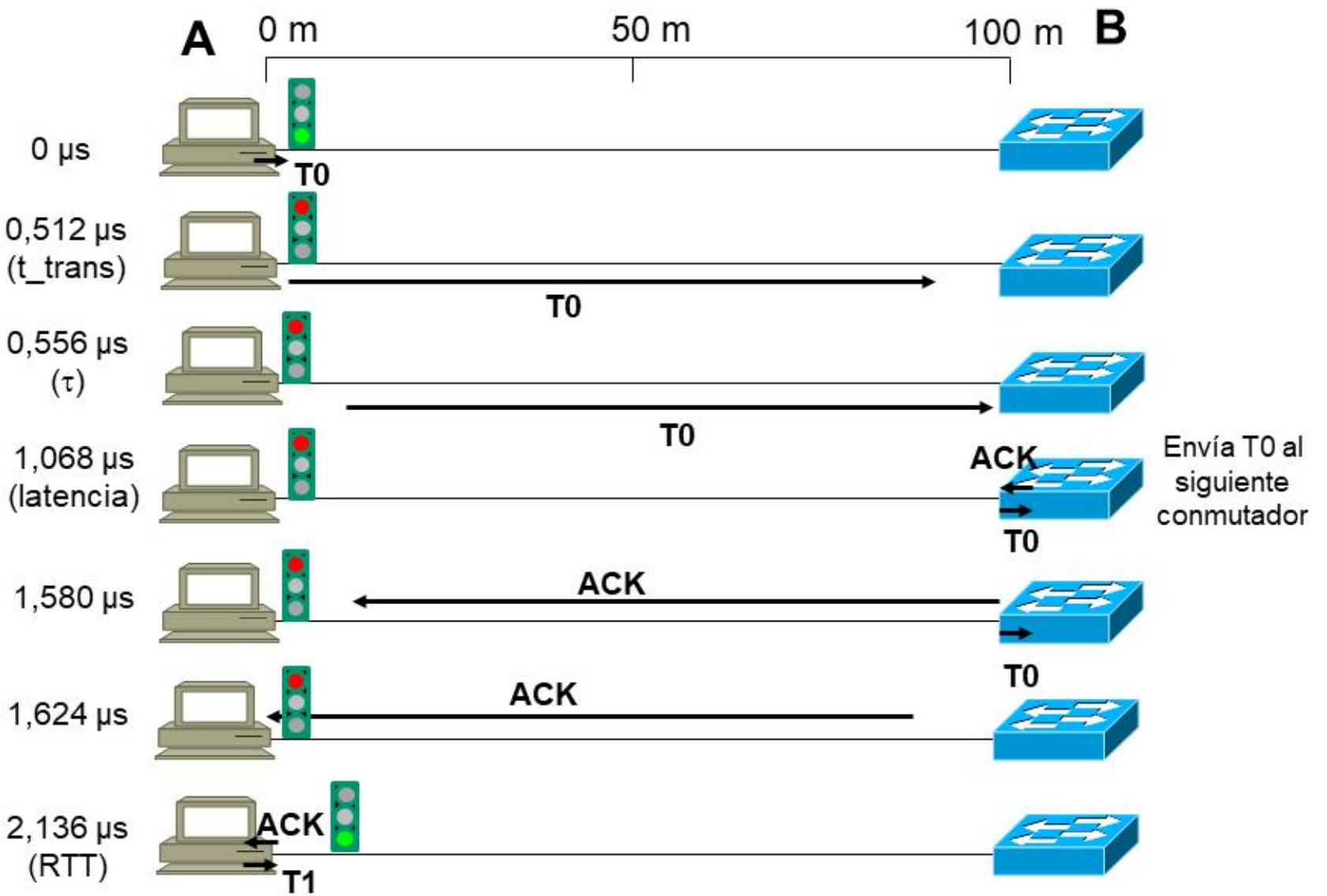


	Distancia en línea recta	RTT	Distancia f.o.
Valencia-Madrid	300 Km	5 ms	500 Km
Madrid-Milán	1200 Km	22 ms	2200 Km
Milán-Ginebra	250 Km	13 ms	1300 Km

Protocolo con control de flujo

- Supongamos ahora que el nivel de red en el receptor no es capaz de digerir los datos al ritmo enviado. En ese caso el buffer se llenará y las tramas se perderán
- Para evitarlo vamos a cambiar el protocolo. Ahora cada vez que enviemos una trama esperaremos una confirmación antes de mandar la siguiente
- La confirmación, o acuse de recibo, es lo que llamamos un ACK (de 'acknowledgement', acuse de recibo en inglés). Si recibimos el ACK enviamos la siguiente trama, si no esperamos. Es como si tuviéramos un semáforo que cuando enviamos una trama se pone en rojo y que al recibir el ACK se pone en verde

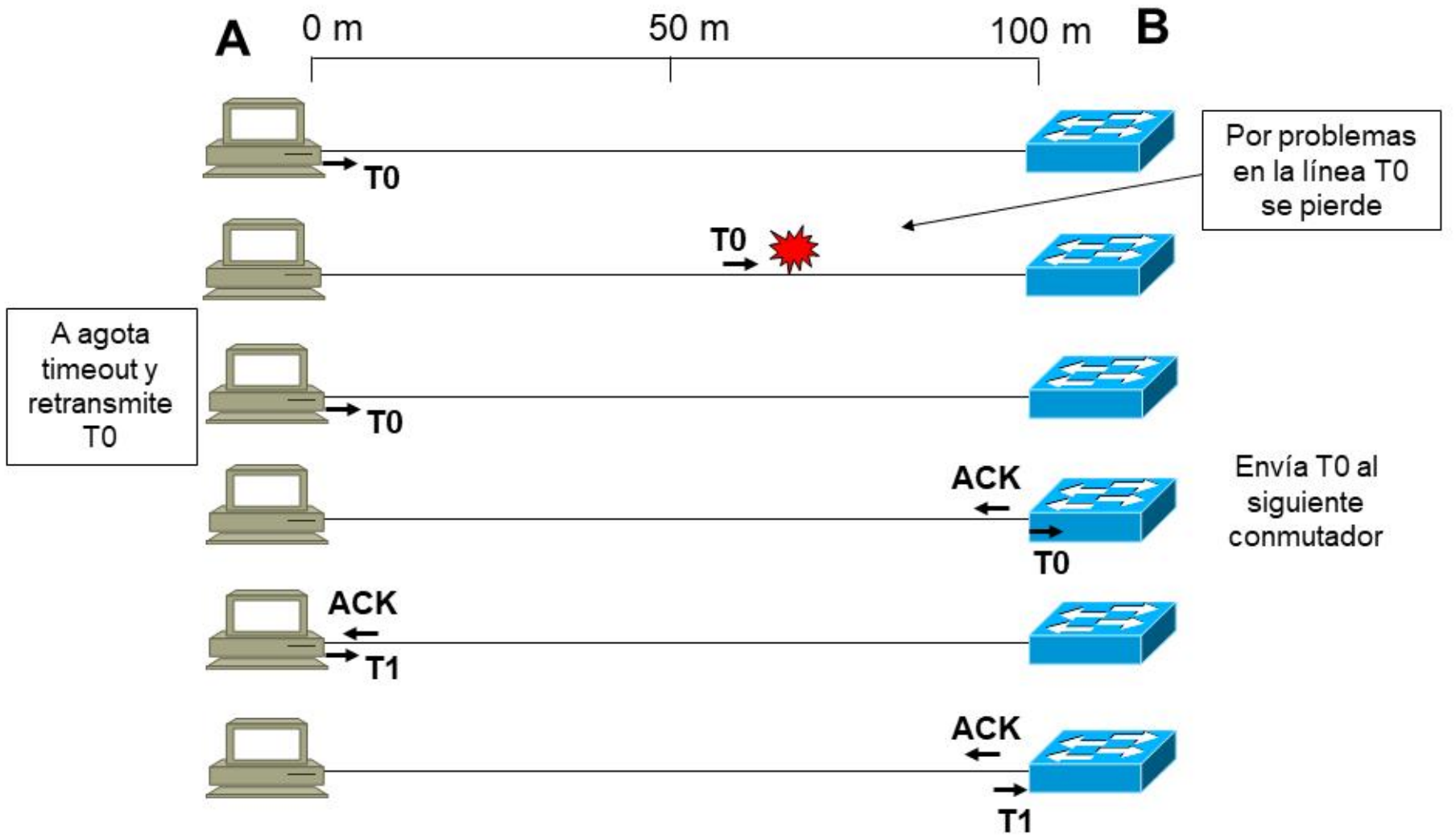
Protocolo con control de flujo



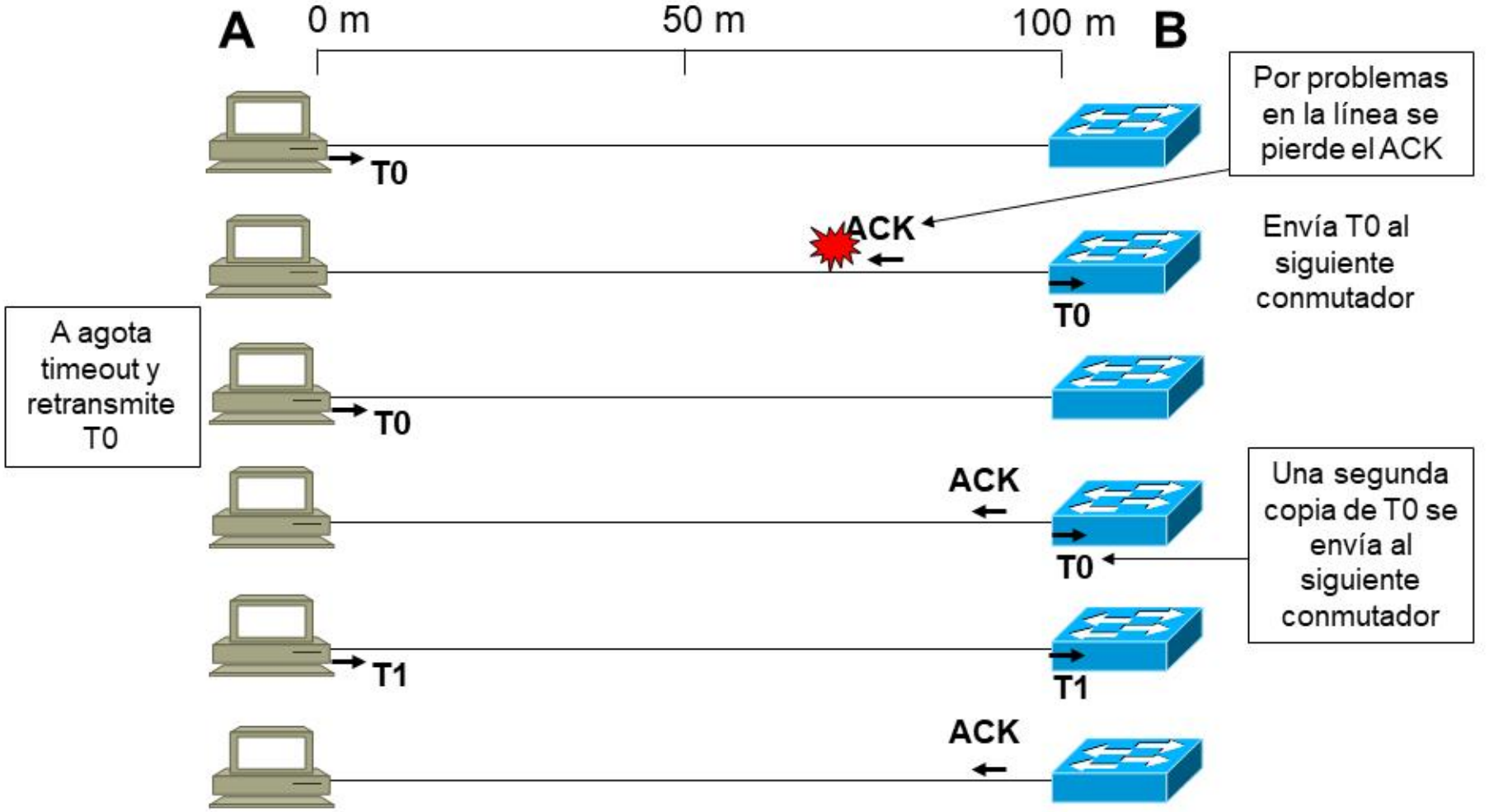
Protocolo resistente a errores I

- Vamos a admitir ahora que pueda haber errores en la transmisión, incluso perderse por completo alguna trama.
- Podemos utilizar el protocolo anterior si convenimos que el ACK indique, no solo espacio disponible en buffers, sino también recepción OK. El emisor reenviará la trama errónea o perdida cuando no reciba el ACK en el plazo de tiempo acordado (timeout).

Protocolo resistente a errores I



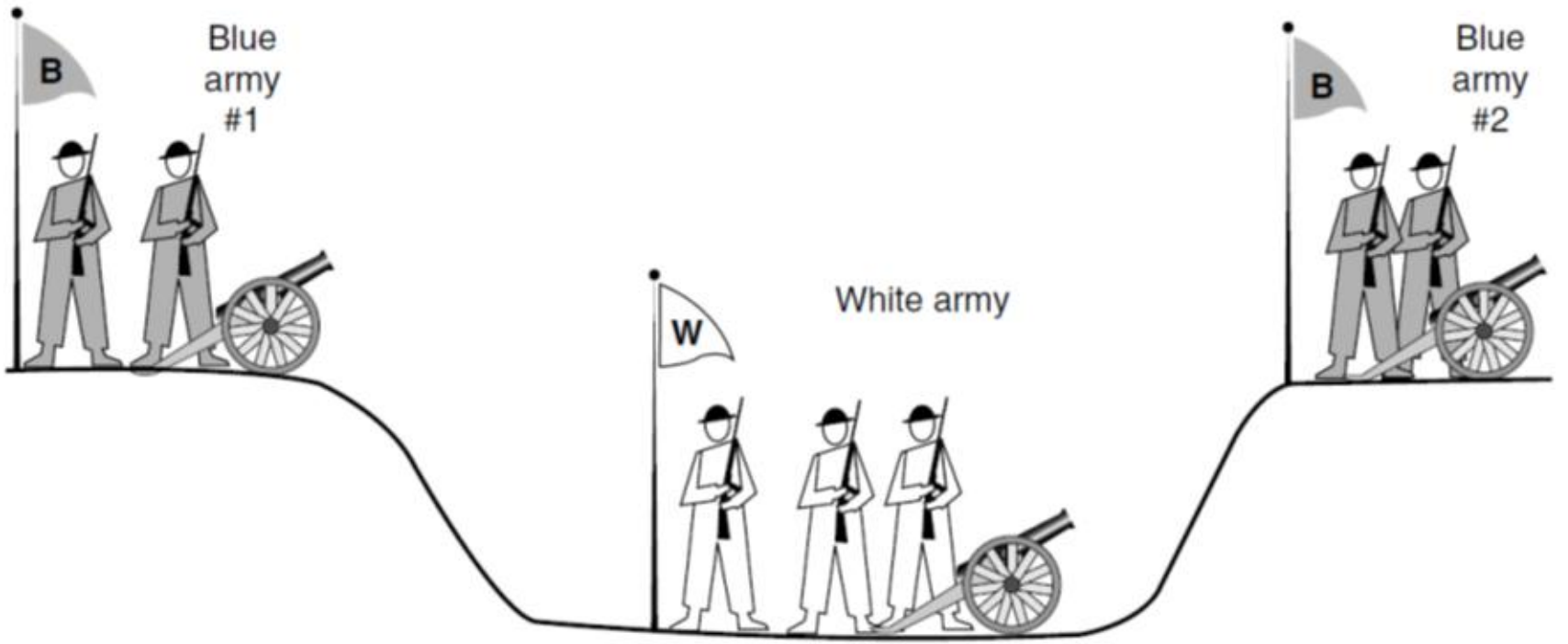
Protocolo resistente a errors I: problema



Posible 'solución' al problema

- Podemos añadir a nuestro protocolo un ACK del ACK, de forma que el receptor sepa cuando se ha perdido su ACK. Pero ¿y si se pierde el ACK del ACK?
- Podemos añadir un ACK del ACK del ACK, y así sucesivamente
- Aparte de la ineficiencia que genera, encadenar ACKs no resuelve el problema. Esto es lo que se conoce como el problema de los dos ejércitos (o de los dos generales)

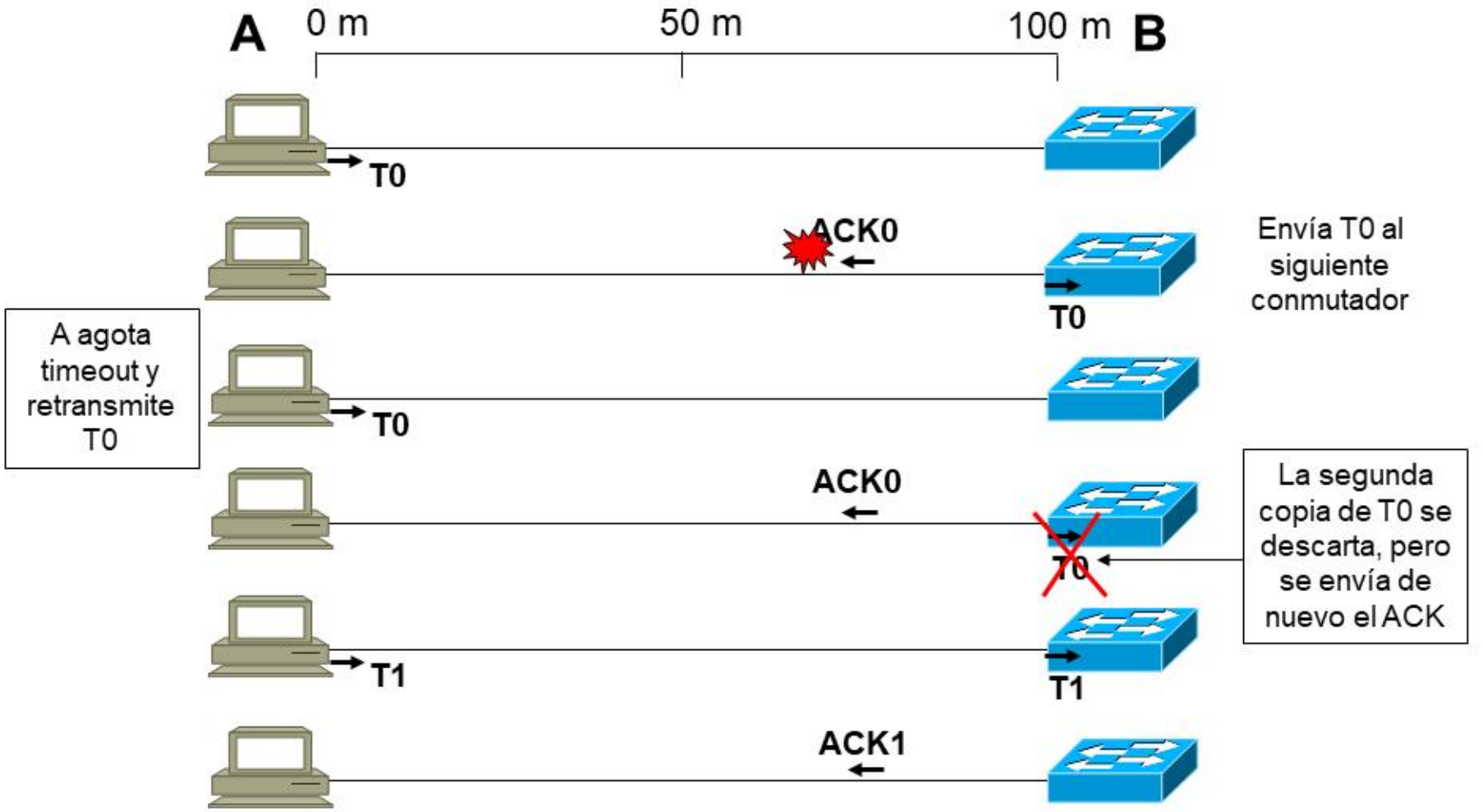
Problema de los dos ejércitos



La verdadera Solución

- El de los dos ejércitos es un problema clásico que no tiene solución, pero el de los ACK sí. Basta con numerar los ACK para asociarlos a las tramas
- En realidad no es preciso un rango de numeración infinito, basta con un bit para numerarlas alternando 0 y 1, ya que solo necesitamos distinguir cada trama (y su ACK) de la que le sigue y la que le precede
- Cuando el receptor reciba una trama duplicada se dará cuenta porque verá que su número coincide con la que acaba de recibir; en ese caso la descartará y repetirá el envío del ACK (de lo contrario el emisor persistirá en el envío indefinidamente)

Protocolo resistente a errores II



Protocolos de parada y espera

- Los ejemplos ejemplos anteriores (protocolo con control de flujo, protocolo resistente a errores I y II) tienen en común la espera del ACK antes de enviar la siguiente trama. Por eso se les denomina 'protocolos de parada y espera'
- En todos ellos el ciclo competo de envío de una trama dura 2,136 μs (un RTT). En este tiempo la línea está ocupada 0,512 μs (t_trans), así que la eficiencia será:

  0,512/2,136 = 0,24 = 24%

Eficiencia en los protocolos de parada y espera

- En general podemos calcular la eficiencia como:

  $$\text{Eficiencia} = t_trans / RTT$$

- Como el ACK suele ser muy pequeño su t_trans se suele considerar despreciable. Si suponemos lo mismo con el tiempo de proceso en destino nos queda que:

  $$RTT = t_trans + 2 * \tau$$

- Así que:

  $$\text{Eficiencia} = t_trans / (t_trans + 2 * \tau)$$

- Si τ es grande en relación con t_trans la eficiencia es muy baja

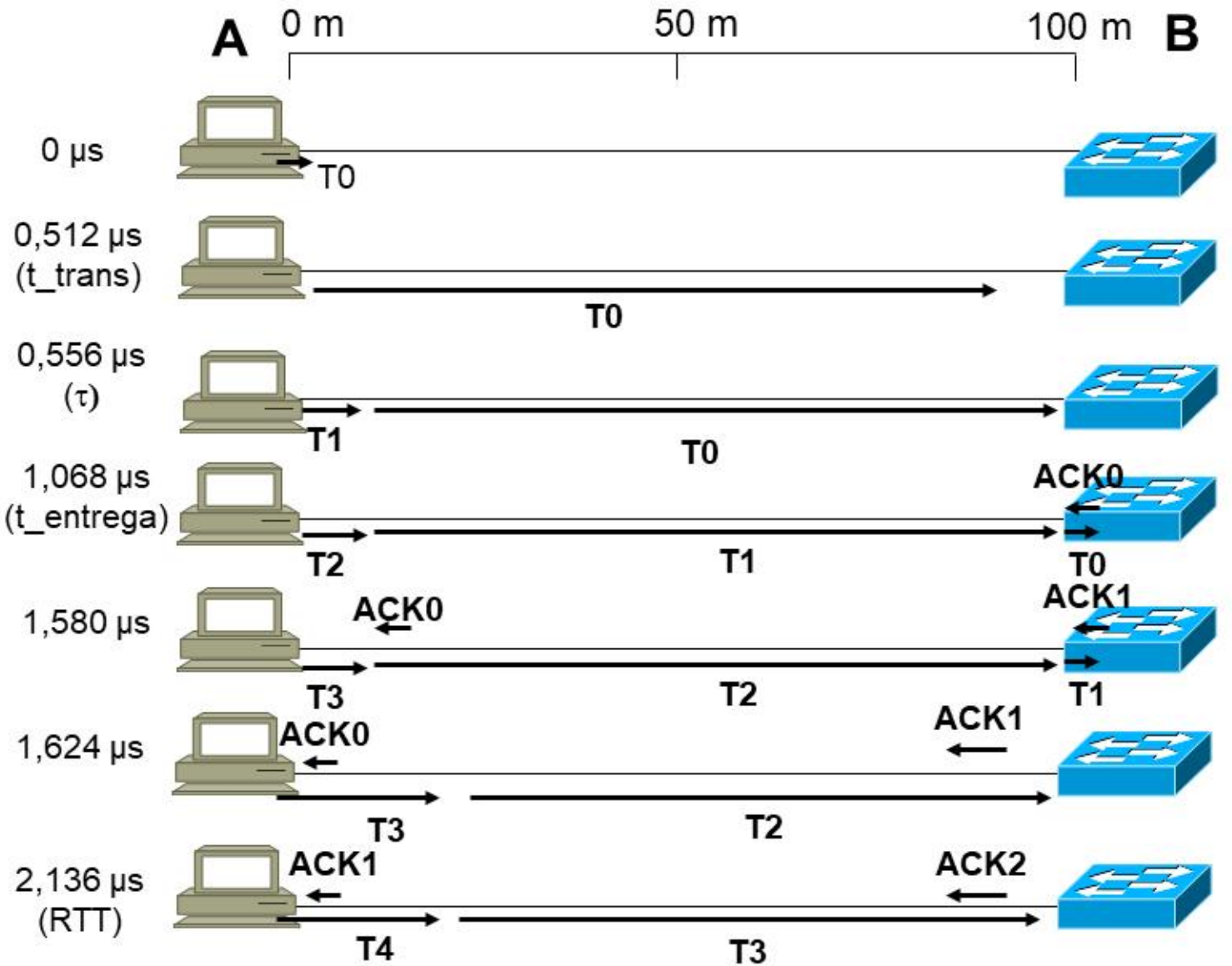
Protocolos de ventana deslizante

- Nuestro protocolo es poco eficiente. Para mejorarlo necesitamos que no pare, y para eso hay que 'llenar' el cable de bits, hacer un 'pipeline'
- Dependiendo de los valores de t_trans y τ el número de tramas necesarias para llenar la tubería puede variar. Solo llenándola podemos conseguir una eficiencia del 100%
- En cada instante podremos tener varias tramas pendientes de confirmación. Es como las tramas fueran una secuencia escrita en una tira de papel tapada y una ventana que se desplaza sobre la tira nos va mostrando las quevan de camino en cada momento

Numeración de las tramas en el protocolo de ventana deslizante

- Como ahora podremos tener varias tramas pendientes de confirmar habrá que numerarlas, y también sus ACKs. Ya no nos valdrá numerar con 0-1, El rango deberá ser mayor que el número máximo de tramas pendientes de confirmar. Así cuando falle alguna sabremos cuál ha sido.
- El número máximo de tramas pendientes de confirmar se conoce como 'tamaño de ventana'

Protocolo de ventana deslizante



Protocolo de ventana deslizante

- Cuando se produce un error hay dos estrategias posibles:
  - **Retroceso-n**: se retransmiten todas las tramas a partir de la errónea
  - **Repetición selectiva**: se retransmite solo la trama errónea y se sigue enviando las tramas como antes del error
- Repetición selectiva es más complejo pero más eficiente. Además requiere más espacio en buffers en el receptor.

Tamaño de ventana

- La ventana mínima para un 100% de ocupación es la que 'llena el hilo' de datos, que se puede calcular con la fórmula:

  W = 2 * τ * caudal / tamaño_trama + 1

  – Ej.: $\tau = 0{,}556\ \mu s$, caudal = 1 Gb/s, tamaño_trama = 512 bits

  $\rightarrow W = 3{,}18 \rightarrow$ Ventana mínima: 4

- Esta fórmula supone:
  – Tiempo de proceso despreciable
  – Longitud de trama ACK cero bytes
  – Mismo τ a la ida que a la vuelta (ruta simétrica)

Tamaño de ventana y número de secuencia

- Para que funcione correctamente un protocol de enlace con ventana deslizante se ha de cumplir que la ventana sea como máximo:
  - En retroceso n: *Número de secuencia – 1*
  - En repetición selectiva: *Número de secuencia/2*
- Por ejemplo, si el número de secuencia es un campo de tres bits:
  - En retroceso-n la ventana máxima es de 7
  - En repetición selectiva la ventana máxima es de 4